

Vulnerability assessment

OpenVAS/Greenbone

Fagnani Marco, Radice Lorenzo

May 19, 2026

Contents

1	Initial configuration	1
1.1	Greenbone	1
1.2	Debian	1
1.2.1	Apache	2
1.2.2	Cups	3
1.2.3	Docker Vulnet	4
1.2.4	FTP	12
1.2.5	Samba	12
1.2.6	SSH	13
1.2.7	Syslog	13
1.3	Windows XP	14
1.4	Network configuration	14
2	System Discovery	17

Abstract

The experience will be focused on the vulnerability assessment of an IT system, using the OpenVAS/Greenbone tool. The main goal is to identify and evaluate potential security weaknesses in a target system.. This process involves scanning the system for known vulnerabilities, analyzing the results, and providing recommendations for remediation. The experience will also cover the configuration and usage of OpenVAS/Greenbone, as well as best practices for conducting vulnerability assessments effectively.

Chapter 1

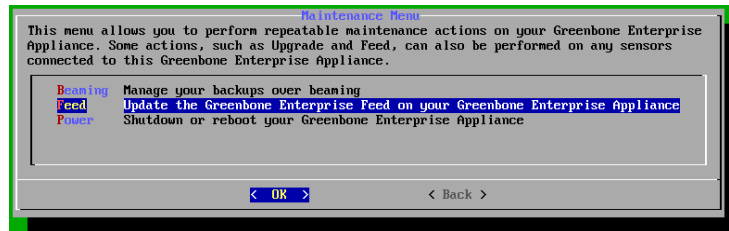
Initial configuration

1.1 Greenbone

Greenbone/OpenVAS is an open-source vulnerability scanning tool that provides comprehensive security assessments for IT systems. To set up Greenbone/OpenVAS, firstly download the official Greenbone/OpenVAS OVA file from the Greenbone website.

<https://www.greenbone.net/en/openvas-free/>

Then, import the OVA file into VirtualBox 6.1 or higher. Assign at least 5GB of RAM and 2 CPU cores to the virtual machine. It's better to use about 12GB of RAM. For the first configuration, use the bridge network adapter, which allows the virtual machine to be on the same network as the host machine. Start the Greenbone/OpenVAS virtual machine and follow the on-screen instructions to complete the initial setup. Assign a user for the web interface. Once the setup is complete, access the Greenbone/OpenVAS update the feed by going to **Maintenance>Feed>Update** and wait it to synchronize (usually hours).



1.2 Debian

Debian is an open-source operating system based on GNU/Linux. It will be used as a base to create a vulnerable machine. We used the latest stable version of Debian, which is Debian 13 Trixie.

1.2.1 Apache

Install Apache web server and create weak certificates for it. configure Apache to allow the use of weak protocols and ciphers, such as SSLv3, TLS1.1, DES, RC4 and MD5.

```
1  #!/bin/bash
2
3  apt-get update
4  apt-get install -y apache2 php libapache2-mod-php
5  rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
6
7  a2enmod ssl
8  a2enmod rewrite
9  a2enmod headers
10 a2enmod proxy
11 a2enmod proxy_http
12 a2dissite 000-default.conf
13 echo "ServerName 127.0.0.1" | tee /etc/apache2/conf-available/servername.conf
14 a2enconf servername
15
16 output_dir=certs
17 mkdir -p $output_dir
18 cd $output_dir
19 openssl genrsa -out weak.key 1024
20 openssl req -new -key weak.key -out weak.csr -subj
21 ↪ "C=XX/ST=Lab/L=Vulnbox/O=Insecure/OU=Testing/CN=localhost"
22 openssl x509 -req -days 1 -in weak.csr -signkey weak.key -out weak.crt -sha1
23 ↪ #openssl x509 -req -days 3650 -in weak.csr -signkey weak.key -out weak_md5.crt
24 ↪ -md5
25
26
27 mkdir -p /etc/apache2/ssl
28 cp weak.* /etc/apache2/ssl/
29
30 tee /etc/apache2/sites-available/vulnbox.conf << EOF
31 <VirtualHost *:80>
32     ServerName localhost
33
34     ProxyPreserveHost On
35     ProxyPass / http://127.0.0.1:8000/
36     ProxyPassReverse / http://127.0.0.1:8000/
37
38     # No security headers
39 </VirtualHost>
40
41 <VirtualHost *:443>
42     ServerName localhost
43
44     SSLEngine on
45     SSLCertificateFile /etc/apache2/ssl/weak.crt
46     SSLCertificateKeyFile /etc/apache2/ssl/weak.key
47
48     # Allow TLS 1.0 only if OpenSSL supports it
49     SSLProtocol all -SSLv3 -TLSv1.3 -TLSv1.2 -TLSv1.1
50     #SSLProtocol all -SSLv3
51
52     SSLCipherSuite ALL:NULL:EXPORT:LOW:MEDIUM:DES:RC4:MD5:@SECLEVEL=0
53     #SSLCipherSuite aNULL:EXPORT:LOW:ALL:@SECLEVEL=0
54     #SSLCipherSuite HIGH:!aNULL:@SECLEVEL=0
```

```

52     #SSLCipherSuite ALL:NULL:EXPORT:LOW:@SECLEVEL=0
53
54     SSLHonorCipherOrder off
55
56     #SSLCompression on
57     SSLSessionTickets On
58
59     SSLOpenSSLConfCmd MinProtocol SSLv3
60     SSLOpenSSLConfCmd CipherString DEFAULT:@SECLEVEL=0
61
62     ProxyPreserveHost On
63     ProxyPass / http://127.0.0.1:8000/
64     ProxyPassReverse / http://127.0.0.1:8000/
65
66     Header always unset Strict-Transport-Security
67     Header always unset X-Frame-Options
68     Header always unset X-Content-Type-Options
69 </VirtualHost>
70 EOF
71
72 a2ensite vulnbox.conf
73
74 tee /etc/php/7.4/apache2/php.ini << EOF
75 display_errors = On
76 display_startup_errors = On
77 allow_url_fopen = On
78 allow_url_include = On
79 disable_functions =
80 open_basedir =
81 expose_php = On
82 EOF
83
84 systemctl restart apache2

```

Any website can be used as exposed service. In our case we used an old project of ours, properly modified to be vulnerable.

```
git clone -b vulnerable https://github.com/ozneroL541/artigianato-online.git
```

To run it execute the following commands in the terminal:

```

1 cd artigianato-online/src
2 docker compose up --remove-orphans -d

```

1.2.2 Cups

Install the CUPS printing system and configure it to allow remote administration without authentication.

```

1 #!/bin/bash
2
3 apt-get update
4 apt-get install -y cups
5 rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
6
7 tee /etc/cups/cupsd.conf << EOF
8 Listen 0.0.0.0:631
9
10 <Location />

```

```

11     Order allow,deny
12     Allow all
13 </Location>
14
15 <Location /admin>
16     Order allow,deny
17     Allow all
18 </Location>
19
20 WebInterface Yes
21 DefaultAuthType Basic
22 EOF
23
24 systemctl enable --now cups

```

1.2.3 Docker Vulnet

To simulate a complex network environment, we are using a docker container with multiple vulnerable services. The scope of this network is to show the capabilities of Greenbone/OpenVAS in scanning a complex system. To set up the network we used the following docker compose file:

```

1 #####
2 # Vulnerability Assessment Lab
3 #
4 # Network: 172.30.0.0/22
5 #   host      - 172.30.0.1/24
6 #   vulnbox1 - 172.30.0.2/24 172.30.1.1/24
7 #   vulnbox2 - 172.30.1.2/24 172.30.2.1/24
8 #   vulnbox3 - 172.30.2.2/24
9 #   vulnbox4 - 172.30.2.3/24
10 #   vulnbox5 - 172.30.2.4/24 172.30.3.1/24
11 #   vulnbox6 - 172.30.3.2/24
12 #####
13
14 networks:
15   # Bridge { directly reachable from host
16   net0:
17     driver: bridge
18     driver_opts:
19       com.docker.network.bridge.name: "vulnlab0"
20     ipam:
21       config:
22         - subnet: 172.30.0.0/24
23           gateway: 172.30.0.1
24
25   # Internal macvlan segments { isolated from host, routed only through vulnbox1
26   net1:
27     driver: bridge
28     driver_opts:
29       com.docker.network.bridge.name: "vulnlab1"
30     internal: true           # blocks direct host - net1 traffic
31     ipam:
32       config:
33         - subnet: 172.30.1.0/24
34           gateway: 172.30.1.254
35

```

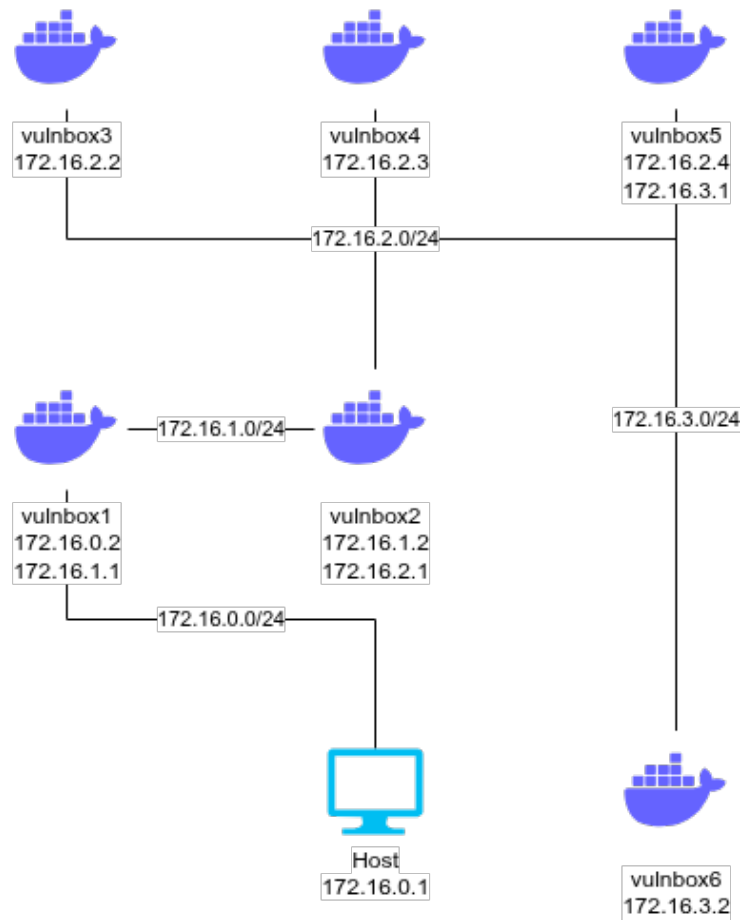


Figure 1.1: Docker network configuration

```

36 net2:
37     driver: bridge
38     driver_opts:
39         com.docker.network.bridge.name: "vulnlab2"
40     internal: true          # blocks direct host - net2 traffic
41     ipam:
42         config:
43             - subnet: 172.30.2.0/24
44               gateway: 172.30.2.254
45
46 net3:
47     driver: bridge
48     driver_opts:
49         com.docker.network.bridge.name: "vulnlab3"
50     internal: true
51     ipam:

```



```

52     config:
53       - subnet: 172.30.3.0/24
54         gateway: 172.30.3.254
55
56
57 services:
58   # --- VULNBOX 1 { Entering node -----
59   vulnbox1:
60     build:
61       context: ./network/vulnbox-1
62       dockerfile: Dockerfile
63     image: vulnbox1:latest
64     container_name: vulnbox1
65     hostname: vulnbox1
66     networks:
67       net0:
68         ipv4_address: 172.30.0.2
69       net1:
70         ipv4_address: 172.30.1.1
71     cap_add:
72       - NET_ADMIN
73     privileged: true
74     restart: unless-stopped
75
76   # --- VULNBOX 2 { Apache Struts 2.3.31 (CVE-2017-5638) ----
77   vulnbox2:
78     build:
79       context: ./network/vulnbox-2
80       dockerfile: Dockerfile
81     image: vulnbox2:latest
82     container_name: vulnbox2
83     hostname: vulnbox2
84     networks:
85       net1:
86         ipv4_address: 172.30.1.2
87       net2:
88         ipv4_address: 172.30.2.1
89     cap_add:
90       - NET_ADMIN
91     privileged: true
92     restart: unless-stopped
93
94   # --- VULNBOX 3 { DVWA -----
95   vulnbox3:
96     build:
97       context: ./network/vulnbox-3
98       dockerfile: Dockerfile
99     image: vulnbox3:latest
100    container_name: vulnbox3
101    hostname: vulnbox3
102    networks:
103      net2:
104        ipv4_address: 172.30.2.2
105    cap_add:
106      - NET_ADMIN
107    privileged: true
108    restart: unless-stopped
109
110   # --- VULNBOX 4 { SSH weak credentials -----

```

```

111 vulnbox4:
112     build:
113         context: ./network/vulnbox-4
114         dockerfile: Dockerfile
115     image: vulnbox4:latest
116     container_name: vulnbox4
117     hostname: vulnbox4
118     networks:
119         net2:
120             ipv4_address: 172.30.2.3
121     cap_add:
122         - NET_ADMIN
123     privileged: true
124     restart: unless-stopped
125
126 # --- VULNBOX 5 { Samba -----
127 vulnbox5:
128     build:
129         context: ./network/vulnbox-5
130         dockerfile: Dockerfile
131     image: vulnbox5:latest
132     container_name: vulnbox5
133     hostname: vulnbox5
134     networks:
135         net2:
136             ipv4_address: 172.30.2.4
137         net3:
138             ipv4_address: 172.30.3.1
139     cap_add:
140         - NET_ADMIN
141     privileged: true
142     restart: unless-stopped
143
144 # --- VULNBOX 6 { MySQL no root password -----
145 vulnbox6:
146     build:
147         context: ./network/vulnbox-6
148         dockerfile: Dockerfile
149     image: vulnbox6:latest
150     container_name: vulnbox6
151     hostname: vulnbox6
152     networks:
153         net3:
154             ipv4_address: 172.30.3.2
155     cap_add:
156         - NET_ADMIN
157     privileged: true
158     restart: unless-stopped

```

To ensure proper network setup it was crucial to set the correct routes through all the containers, so that they could communicate with each other in the way we wanted. To do so we wrote the following script:

```

1  #!/bin/bash
2  # Stop and remove any existing containers
3  docker compose down --remove-orphans
4  # Start the vulnbox network
5  docker compose up --remove-orphans -d
6  # Allow forwarding between containers

```

```

7 iptables -I FORWARD 1 -s 172.30.0.0/22 -d 172.30.0.0/22 -j ACCEPT
8 # Remove the default IP addresses from the bridge interfaces
9 ip addr del 172.30.1.254/24 dev vulnlab1
10 ip addr del 172.30.2.254/24 dev vulnlab2
11 ip addr del 172.30.3.254/24 dev vulnlab3
12 # Host
13 # Tell your physical machine to send lab traffic to vulnbox1
14 ip route replace 172.30.1.0/24 via 172.30.0.2
15 ip route replace 172.30.2.0/24 via 172.30.0.2
16 ip route replace 172.30.3.0/24 via 172.30.0.2
17 # vulnbox1
18 docker exec -u 0 vulnbox1 sysctl -w net.ipv4.ip_forward=1
19 docker exec -u 0 vulnbox1 ip route replace 172.30.2.0/24 via 172.30.1.2
20 docker exec -u 0 vulnbox1 ip route replace 172.30.3.0/24 via 172.30.1.2
21 docker exec -u 0 vulnbox1 ip route add default via 172.30.0.1
22 # vulnbox2
23 docker exec -u 0 vulnbox2 sysctl -w net.ipv4.ip_forward=1
24 docker exec -u 0 vulnbox2 ip route replace 172.30.3.0/24 via 172.30.2.4
25 docker exec -u 0 vulnbox2 ip route add default via 172.30.1.1
26 # vulnbox3
27 docker exec -u 0 vulnbox3 ip route add 172.30.3.0/24 via 172.30.2.4
28 docker exec -u 0 vulnbox3 ip route add default via 172.30.2.1
29 # vulnbox4
30 docker exec -u 0 vulnbox4 ip route replace 172.30.3.0/24 via 172.30.2.4
31 docker exec -u 0 vulnbox4 ip route add default via 172.30.2.1
32 # vulnbox5
33 docker exec -u 0 vulnbox5 sysctl -w net.ipv4.ip_forward=1
34 docker exec -u 0 vulnbox5 ip route add default via 172.30.2.1
35 # vulnbox6
36 docker exec -u 0 vulnbox6 ip route add default via 172.30.3.1

```

To let Greenbone/OpenVAS scan the containers, we had to create further rules. Since the VM is intended as a vulnerable machine, we had no intention to let the firewall block any traffic, therefore, firstly, we flushed all its rules and then we build up our own ruleset, which allows all the traffic from the Greenbone/OpenVAS to the container's network.

```

1 #!/bin/bash
2
3 # DO NOT RUN THIS ON YOUR HOST MACHINE
4 # VM ONLY
5
6 docker_interface="vulnlab0"
7 docker_subnet="172.30.0.0/22"
8 interface=$(ip -o -f inet addr show | awk '/scope global/ {print $2}' | head -n
↪ 1)
9
10 ./setup.sh
11 # Flush everything and start clean
12 iptables -F FORWARD
13 iptables -t nat -F POSTROUTING
14 nft flush chain ip raw PREROUTING
15
16 # Allow forwarding between containers
17 iptables -I FORWARD 1 -s $docker_subnet -d $docker_subnet -j ACCEPT
18
19 # Re-add Docker's default NAT rules
20 iptables -t nat -A POSTROUTING -s $docker_subnet ! -o $docker_interface -j
↪ MASQUERADE

```

```

21 iptables -t nat -A POSTROUTING -s 172.17.0.0/16 ! -o docker0 -j MASQUERADE
22
23 # Allow forwarding between eth and bridge
24 iptables -I FORWARD 1 -i $interface -o $docker_interface -j ACCEPT
25 iptables -I FORWARD 1 -i $docker_interface -o $interface -j ACCEPT
26
27 # Allow Docker chain
28 iptables -I DOCKER 1 -i $interface -d $docker_subnet -j ACCEPT

```

vulnbox1

The first vulnbox was intended to be vulnerable to log4Shell (CVE-2021-44228), a critical vulnerability in the Apache Log4j library. However, since the container was handmade, Greenbone/OpenVAS was not able to detect it.

```

1  # Vulnbox 1 - Log4Shell
2  # Pivot node for networking
3  FROM maven:3.9.9-eclipse-temurin-8 AS build
4
5  WORKDIR /build
6  COPY pom.xml .
7  RUN mvn -q -e -DskipTests
   ↪ org.apache.maven.plugins:maven-dependency-plugin:3.3.0:go-offline
8
9  COPY src ./src
10 RUN mvn -q -DskipTests package
11
12 # ---- runtime ----
13 FROM eclipse-temurin:8-jre-jammy
14
15 # install tools for your lab
16 RUN apt-get update -o Acquire::AllowInsecureRepositories=true \
17     -o Acquire::AllowDowngradeToInsecureRepositories=true && \
18     apt-get install -y --allow-unauthenticated iproute2 curl && \
19     apt-get clean && \
20     rm -rf /var/lib/apt/lists/* /var/cache/apt/archives/*
21
22 WORKDIR /app
23 COPY --from=build /build/target/vulnapp-1.0-jar-with-dependencies.jar .
24
25 EXPOSE 8080
26
27 CMD ["java", "-jar", "vulnapp-1.0-jar-with-dependencies.jar"]

```

Even if the vulnerability was not detected, the main role of the first vulnbox was to route all the traffic to the docker network and vice versa and, in that sense, it was successful.

vulnbox2

The second vulnbox presents a vulnerable version of the Apache Struts framework, which is affected by the CVE-2017-5638 vulnerability.

```

1  # Apache Struts 2.3.31 { CVE-2017-5638
2  # The Content-Type header is parsed by the Jakarta Multipart parser.
3  # A malformed value triggers OGNL expression evaluation → unauthenticated RCE.

```

```

4  # This is the exact vulnerability used in the 2017 Equifax breach.
5
6  FROM ubuntu:20.04
7
8  ENV DEBIAN_FRONTEND=noninteractive
9
10 # Install Java 8 + Tomcat 8 + wget
11 RUN apt-get update -qq && \
12     apt-get install -y -qq \
13         openjdk-8-jdk-headless \
14         wget \
15         curl \
16         unzip \
17         iproute2 && \
18     rm -rf /var/lib/apt/lists/*
19
20 ENV JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
21 ENV PATH="$JAVA_HOME/bin:$PATH"
22
23 # -- Tomcat 8.5 -----
24 ENV TOMCAT_VERSION=8.5.87
25 ENV CATALINA_HOME=/opt/tomcat
26
27 RUN wget -q
28     ↪ "https://archive.apache.org/dist/tomcat/tomcat-8/v${TOMCAT_VERSION}/bin/apache-tomcat-${TOMCAT_VERSION}.tar.gz" \
29     ↪ \
30         -O /tmp/tomcat.tar.gz && \
31     mkdir -p "$CATALINA_HOME" && \
32     tar xzf /tmp/tomcat.tar.gz -C "$CATALINA_HOME" --strip-components=1 && \
33     rm /tmp/tomcat.tar.gz
34
35 # -- Struts 2.3.31 showcase WAR (contains the vulnerable REST plugin) --
36 ENV STRUTS_VERSION=2.3.31
37
38 RUN wget -q
39     ↪ "https://archive.apache.org/dist/struts/${STRUTS_VERSION}/struts-${STRUTS_VERSION}-apps.zip" \
40     ↪ \
41         -O /tmp/struts-apps.zip && \
42     unzip -q /tmp/struts-apps.zip
43     ↪ "struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
44     ↪ -d /tmp/struts-extract/ && \
45     cp "/tmp/struts-extract/struts-${STRUTS_VERSION}/apps/struts2-showcase.war" \
46     "${CATALINA_HOME}/webapps/struts2-showcase.war" && \
47     rm -rf /tmp/struts-apps.zip /tmp/struts-extract
48
49 # Remove default Tomcat apps to keep the attack surface clean/focused
50 RUN rm -rf \
51     "$CATALINA_HOME/webapps/ROOT" \
52     "$CATALINA_HOME/webapps/docs" \
53     "$CATALINA_HOME/webapps/examples" \
54     "$CATALINA_HOME/webapps/host-manager" \
55     "$CATALINA_HOME/webapps/manager"
56
57 # Tomcat manager creds (useful for post-exploit exploration)
58 COPY tomcat-users.xml "$CATALINA_HOME/conf/tomcat-users.xml"
59
60 EXPOSE 8080
61
62

```

```
58 CMD ["sh", "-c", "$CATALINA_HOME/bin/catalina.sh run"]
```

vulnbox3

The third vulnbox is based on DVWA (Damn Vulnerable Web Application), a web application that is intentionally vulnerable to various types of attacks. We had no clue if Greenbone/OpenVAS would have been able to detect any vulnerabilities here, but since it's in fact the scope of such experiment, we decided to include it.

```
1 # Vulnbox 3 { DVWA
2 # Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web app designed for
  ↳ security training.
3 # It contains multiple vulnerabilities of varying difficulty, making it ideal for
  ↳ practice.
4
5 FROM vulnerables/web-dvwa
6
7 ENV MYSQL_PASS=p@ssw0rd
8 ENV HOSTNAME=vulnbox3
9
10
11 EXPOSE 80 443
```

vulnbox4

The fourth vulnbox is a simple Ubuntu 22.04 image with weak SSH credentials.

```
1 # Vulnbox 4 - SSH Service with Weak Credentials
2 # This container runs an SSH server with weak credentials for testing purposes.
3 FROM ubuntu:22.04
4
5 RUN apt-get update -qq && \
6     apt-get install -y -qq openssh-server iproute2 && \
7     rm -rf /var/lib/apt/lists/* && \
8     useradd -m -s /bin/bash admin; \
9     echo 'admin:admin123' | chpasswd; \
10    useradd -m -s /bin/bash user; \
11    echo 'user:password' | chpasswd; \
12    mkdir -p /run/sshd && \
13    sed -i 's/#PermitRootLogin.*/PermitRootLogin yes/' /etc/ssh/sshd_config && \
14    sed -i 's/#PasswordAuthentication.*/PasswordAuthentication yes/'
  ↳ /etc/ssh/sshd_config && \
15    echo 'root:toor' | chpasswd
16
17 EXPOSE 22
18
19
20 CMD ["/usr/sbin/sshd", "-D"]
```

vulnbox5

The fifth vulnbox is a weak Samba server.

```
1 # Vulnbox 5 - Samba (EternalBlue-era misconfiguration)
```

```

2  # This container runs a Samba server with a misconfiguration that allows for
   ↪ potential exploitation.
3
4  FROM dperson/samba
5
6  RUN mkdir -p /public /private && \
7      chmod 777 /public && \
8      chmod 700 /private
9
10 ENV USERID=1000 \
11     GROUPID=1000
12
13
14 CMD ["-u", "guest;guest", \
15     "-s", "public;/public;yes;no;yes;all;none", \
16     "-s", "private;/private;no;no;no;all;none"]

```

vulnbox6

The sixth vulnbox is a MySQL server with no password.

```

1  FROM mysql:5.7
2
3  ENV MYSQL_ALLOW_EMPTY_PASSWORD=yes
4
5  RUN yum clean all && \
6      yum install -y iproute && \
7      yum clean all && \
8      rm -rf /var/cache/yum

```

1.2.4 FTP

The FTP server is a simple instance of `vsftpd` with multiple weak configurations, such as:

```

1  local_enable=YES
2  write_enable=YES
3  pam_service_name=vsftpd

```

1.2.5 Samba

The Samba server is configured to allow insecure access to a shared folder.

```

1  #!/bin/bash
2
3  apt-get install -y samba
4
5  tee /etc/samba/smb.conf << EOF
6  [global]
7      workgroup = WORKGROUP
8      server string = Vuln Samba
9      map to guest = Bad User
10     guest account = nobody
11
12     # Weak / legacy protocol support
13     server min protocol = NT1

```

```

14     ntlm auth = yes
15     lanman auth = yes
16
17     # Disable signing
18     server signing = disabled
19
20 [vulnshare]
21     path = /srv/samba/vuln
22     browsable = yes
23     writable = yes
24     guest ok = yes
25     read only = no
26     force user = nobody
27 EOF
28
29 mkdir -p /srv/samba/vuln
30 chmod 777 /srv/samba/vuln
31 systemctl enable --now smbd

```

1.2.6 SSH

The SSH server is configured with weak configurations, such as:

```

1 PermitRootLogin yes
2 PasswordAuthentication no

```

1.2.7 Syslog

The Syslog server is configured to allow remote logging without authentication on port 514.

```

1  #!/bin/bash
2
3  #apt-get update
4  #apt-get install -y rsyslog
5  #apt-get clean
6  #rm -rf /var/lib/apt/lists/*
7
8  # Enable UDP + TCP syslog reception
9  cat << EOF > /etc/rsyslog.d/10-remote.conf
10 # Load modules
11 module(load="imudp")
12 input(type="imudp" port="514")
13
14 module(load="imtcp")
15 input(type="imtcp" port="514")
16
17 # Log EVERYTHING from remote hosts
18 *.* /var/log/remote.log
19
20 # No rate limiting (intentionally bad)
21 \$$SystemLogRateLimitInterval 0
22 EOF
23
24 # Ensure rsyslog is enabled
25 systemctl enable rsyslog
26 systemctl restart rsyslog

```



```

27
28 mkdir -p /var/log/
29 chmod 777 -R /var/log/

```

1.3 Windows XP

Windows XP is an old operating system that is no longer supported by Microsoft, which makes it vulnerable to various types of attacks. Just its mere presence in the network is a vulnerability, since it can be easily exploited by attackers. To set up Windows XP, we just installed its virtual machine on VirtualBox and connected it to the same network of the other machines.

1.4 Network configuration

The whole network configuration is an Host-only virtual network created with VirtualBox, which allows all the machines to communicate with each other and have no access to other networks, such as the Internet.

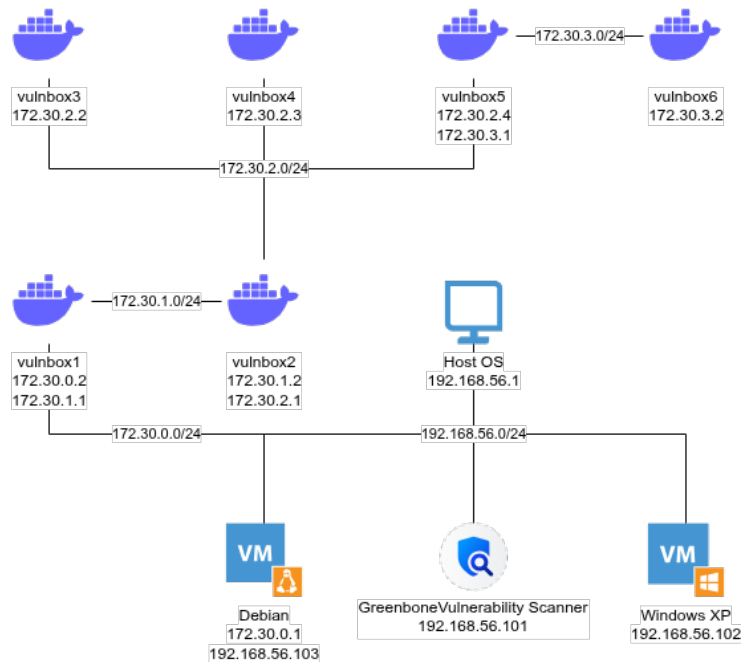
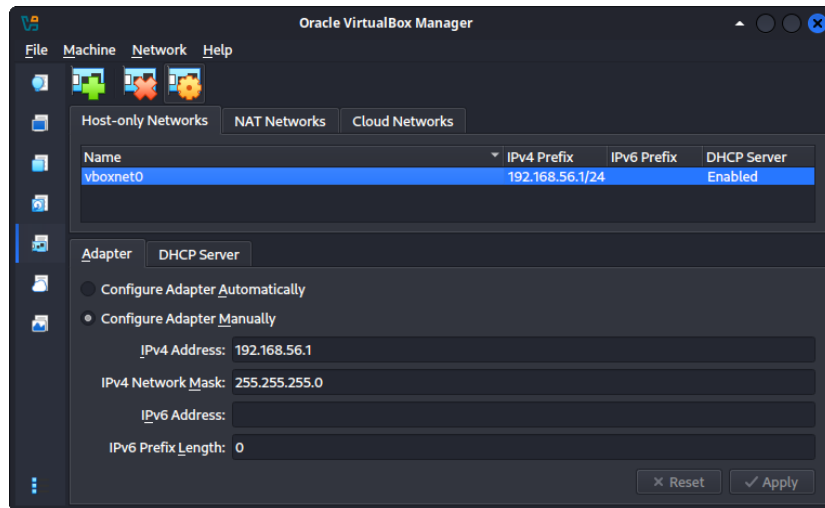


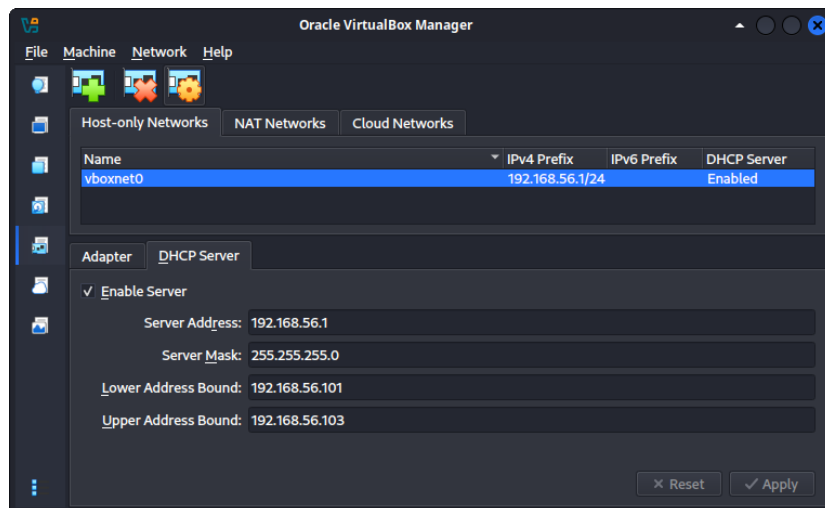
Figure 1.2: Full laboratory's network

In order to create the virtual network and connect all the machines to it, we followed the following procedure:

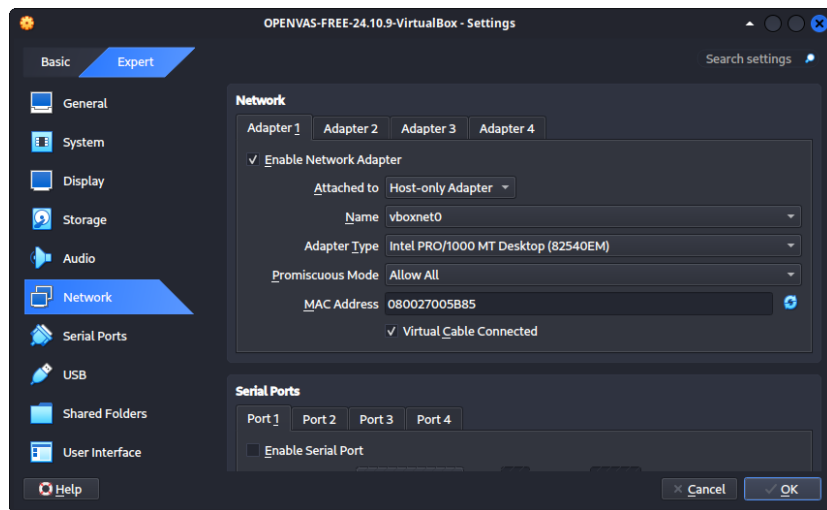
1. Create a Host-only network in VirtualBox



2. Enable the DHCP server for the Host-only network enough to assign IP addresses to all the 3 machines



3. Go to settings of each machine and connect it to the Host-only network



Chapter 2

System Discovery